Context

Jamboree has helped thousands of students secure admissions to top colleges abroad. Whether it is **GMAT, GRE, or SAT**, their unique problem-solving methods ensure maximum scores with minimum effort.

Recently, Jamboree launched a feature on their website that allows students/learners to check their **probability of getting into Ivy League colleges**.
This tool is designed to estimate the chances of graduate admission from an **Indian student's perspective**.

Column Profiling

- **Serial No.** → Unique row identifier
- **GRE Score** → Graduate Record Examination score (out of 340)
- **TOEFL Score** → Test of English as a Foreign Language score (out of 120)
- **University Rating** → Rating of the university (scale of 1–5)
- **SOP & LOR Strength** → Quality of Statement of Purpose and Letters of Recommendation (scale of 1–5)
- **Undergraduate GPA (CGPA)** → Academic performance in undergrad (out of 10)
- **Research Experience** → Whether the student has prior research experience (**0 = No, 1 = Yes**)

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats
```

In [5]:
```
df = pd.read_csv("Jamboree_Admission.csv")
df.head()
```

Out[5]:

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65

In [6]:
```
df.tail()
```

Out[6]:

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
495	496	332	108	5	4.5	4.0	9.02	1	0.87
496	497	337	117	5	5.0	5.0	9.87	1	0.96
497	498	330	120	5	4.5	5.0	9.56	1	0.93
498	499	312	103	4	4.0	5.0	8.43	0	0.73
499	500	327	113	4	4.5	4.5	9.04	0	0.84

In [7]:
```
df.shape
```

Out[7]:
```
(500, 9)
```

In [8]:
```
df.describe().T
```

Out[8]:

	count	mean	std	min	25%	50%	75%	max
Serial No.	500.0	250.50000	144.481833	1.00	125.7500	250.50	375.25	500.00
GRE Score	500.0	316.47200	11.295148	290.00	308.0000	317.00	325.00	340.00
TOEFL Score	500.0	107.19200	6.081868	92.00	103.0000	107.00	112.00	120.00
University Rating	500.0	3.11400	1.143512	1.00	2.0000	3.00	4.00	5.00
SOP	500.0	3.37400	0.991004	1.00	2.5000	3.50	4.00	5.00
LOR	500.0	3.48400	0.925450	1.00	3.0000	3.50	4.00	5.00
CGPA	500.0	8.57644	0.604813	6.80	8.1275	8.56	9.04	9.92
Research	500.0	0.56000	0.496884	0.00	0.0000	1.00	1.00	1.00
Chance of Admit	500.0	0.72174	0.141140	0.34	0.6300	0.72	0.82	0.97

In [9]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Serial No.         500 non-null    int64
 1   GRE Score          500 non-null    int64
 2   TOEFL Score        500 non-null    int64
 3   University Rating  500 non-null    int64
 4   SOP                500 non-null    float64
 5   LOR                500 non-null    float64
 6   CGPA               500 non-null    float64
 7   Research           500 non-null    int64
 8   Chance of Admit    500 non-null    float64
dtypes: float64(4), int64(5)
memory usage: 35.3 KB
```

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
Serial No.           0
GRE Score            0
TOEFL Score          0
University Rating    0
SOP                  0
LOR                  0
CGPA                 0
Research             0
Chance of Admit      0
dtype: int64
```

In [11]:
```
df[df.duplicated()].count()
```

Out[11]:
```
Serial No.           0
GRE Score            0
TOEFL Score          0
University Rating    0
SOP                  0
LOR                  0
CGPA                 0
Research             0
Chance of Admit      0
dtype: int64
```

Dataset Overview

- The dataset contains **500 records** and **9 columns**.
- **Serial No.** column consists of unique identifier values.
- There are **no null values** present in the dataset.
- There are **no duplicate records** found in the dataset.

In [12]:
```
df.drop(columns=['Serial No.'], inplace=True)
```

In [13]:
```
df.columns
```

Out[13]:
```
Index(['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ', 'CGPA',
       'Research', 'Chance of Admit '],
      dtype='object')
```

In [14]:
```
df.columns = [i.strip() for i in df.columns]
df.columns = [col.replace(" ", "_") for col in df.columns]
df.columns = [col.lower() for col in df.columns]
df.columns
```

Out[14]:
```
Index(['gre_score', 'toefl_score', 'university_rating', 'sop', 'lor', 'cgpa',
       'research', 'chance_of_admit'],
      dtype='object')
```

In [15]:
```
cat_cols = ['university_rating', 'sop', 'lor', 'research']
num_cols = ["gre_score", 'toefl_score', 'cgpa']
target = "chance_of_admit"
```

In [16]:
```
for col in df.columns:
    print(col)
    print("the count of unique values are:", df[col].nunique())
    print("The maximum value in the col is: ", df[col].max())
    print("The minimum value in the col is: ", df[col].min())
    print("The count of unique values in the columns are: ", df[col].value_counts(n
    print("_"*150)
```

```
gre_score
the count of unique values are: 49
The maximum value in the col is:  340
The minimum value in the col is:  290
The count of unique values in the columns are:  gre_score
312    0.048
324    0.046
316    0.036
322    0.034
327    0.034
321    0.034
311    0.032
320    0.032
314    0.032
325    0.030
317    0.030
308    0.026
323    0.026
315    0.026
300    0.024
319    0.024
313    0.024
326    0.024
304    0.024
318    0.024
305    0.022
310    0.022
301    0.022
307    0.020
329    0.020
299    0.020
298    0.020
331    0.018
328    0.018
309    0.018
340    0.018
330    0.016
334    0.016
332    0.016
302    0.014
306    0.014
297    0.012
296    0.010
303    0.010
336    0.010
295    0.010
333    0.008
338    0.008
335    0.008
339    0.006
337    0.004
294    0.004
290    0.004
293    0.002
Name: proportion, dtype: float64
```

```
_____________________________________________________________
toefl_score
the count of unique values are: 29
The maximum value in the col is:  120
The minimum value in the col is:  92
The count of unique values in the columns are:  toefl_score
110    0.088
105    0.074
104    0.058
106    0.056
107    0.056
112    0.056
103    0.050
100    0.048
102    0.048
99     0.046
111    0.040
101    0.040
108    0.038
109    0.038
113    0.038
114    0.036
116    0.032
115    0.022
118    0.020
119    0.020
98     0.020
120    0.018
117    0.016
97     0.014
96     0.012
95     0.006
93     0.004
94     0.004
92     0.002
Name: proportion, dtype: float64
_____________________________________________________________________________________________
_____________________________________________________________
university_rating
the count of unique values are: 5
The maximum value in the col is:  5
The minimum value in the col is:  1
The count of unique values in the columns are:  university_rating
3    0.324
2    0.252
4    0.210
5    0.146
1    0.068
Name: proportion, dtype: float64
_____________________________________________________________________________________________
_____________________________________________________________
sop
the count of unique values are: 9
The maximum value in the col is:  5.0
The minimum value in the col is:  1.0
The count of unique values in the columns are:  sop
```

```
4.0    0.178
3.5    0.176
3.0    0.160
2.5    0.128
4.5    0.126
2.0    0.086
5.0    0.084
1.5    0.050
1.0    0.012
Name: proportion, dtype: float64
```

```
lor
the count of unique values are: 9
The maximum value in the col is:  5.0
The minimum value in the col is:  1.0
The count of unique values in the columns are:  lor
3.0    0.198
4.0    0.188
3.5    0.172
4.5    0.126
2.5    0.100
5.0    0.100
2.0    0.092
1.5    0.022
1.0    0.002
Name: proportion, dtype: float64
```

```
cgpa
the count of unique values are: 184
The maximum value in the col is:  9.92
The minimum value in the col is:  6.8
The count of unique values in the columns are:  cgpa
8.00    0.018
8.76    0.018
8.56    0.014
8.12    0.014
8.45    0.014
        ...
7.57    0.002
7.21    0.002
9.27    0.002
7.81    0.002
7.69    0.002
Name: proportion, Length: 184, dtype: float64
```

```
research
the count of unique values are: 2
The maximum value in the col is:  1
The minimum value in the col is:  0
The count of unique values in the columns are:  research
1    0.56
0    0.44
Name: proportion, dtype: float64
```

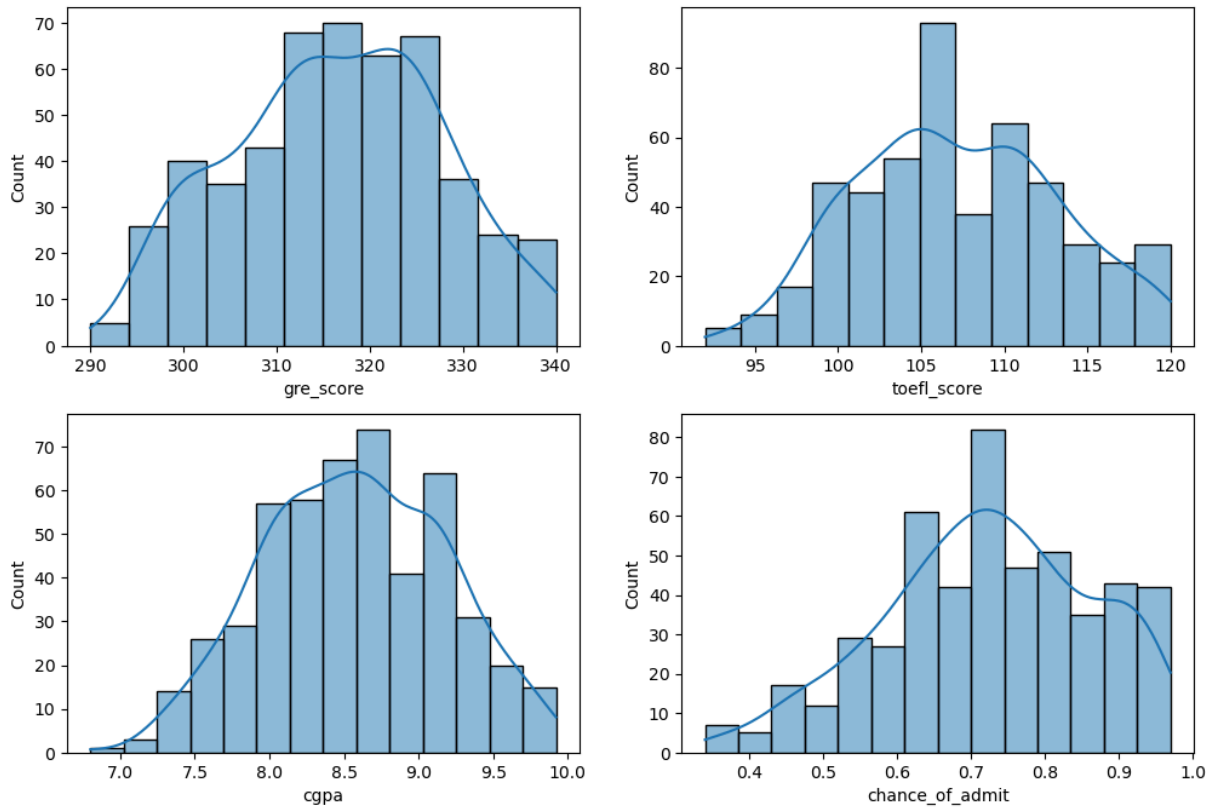
```
chance_of_admit
the count of unique values are: 61
The maximum value in the col is:  0.97
The minimum value in the col is:  0.34
The count of unique values in the columns are:  chance_of_admit
0.71    0.046
0.64    0.038
0.73    0.036
0.72    0.032
0.79    0.032
        ...
0.60    0.004
0.51    0.004
0.43    0.002
0.39    0.002
0.37    0.002
Name: proportion, Length: 61, dtype: float64
```

In [ ]:

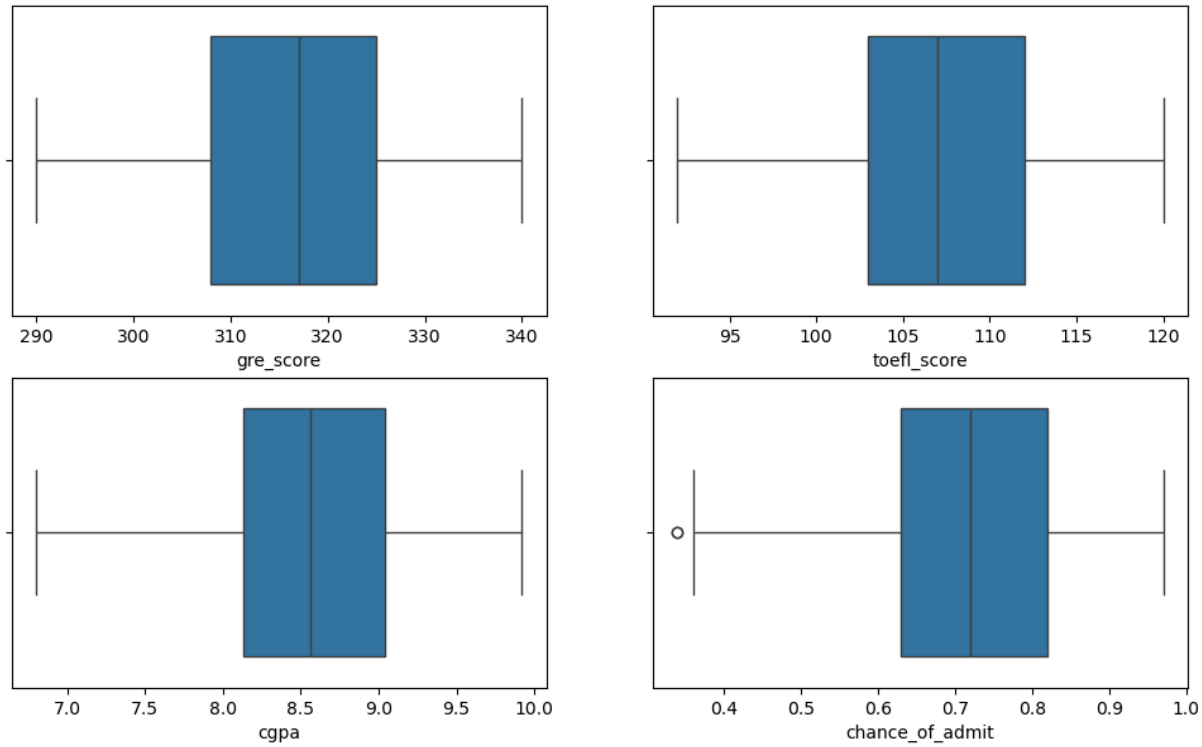
Univariate Analysis

In [17]:

```
# check distribution of each numerical variable
rows, cols = 2, 2
fig, axs = plt.subplots(rows,cols, figsize=(12, 8))
index = 0
for row in range(rows):
    for col in range(cols):
        sns.histplot(df[num_cols[index]], kde=True, ax=axs[row,col])
        index += 1
    break

sns.histplot(df[num_cols[-1]], kde=True, ax=axs[1,0])
sns.histplot(df[target], kde=True, ax=axs[1,1])
plt.show()
```



In [18]:
```
# check for outliers using boxplots
rows, cols = 2, 2
fig, axs = plt.subplots(rows, cols, figsize=(12, 7))

index = 0
for col in range(cols):
    sns.boxplot(x=num_cols[index], data=df, ax=axs[0,index])
    index += 1

sns.boxplot(x=num_cols[-1], data=df, ax=axs[1,0])
sns.boxplot(x=target, data=df, ax=axs[1,1])
plt.show()
```



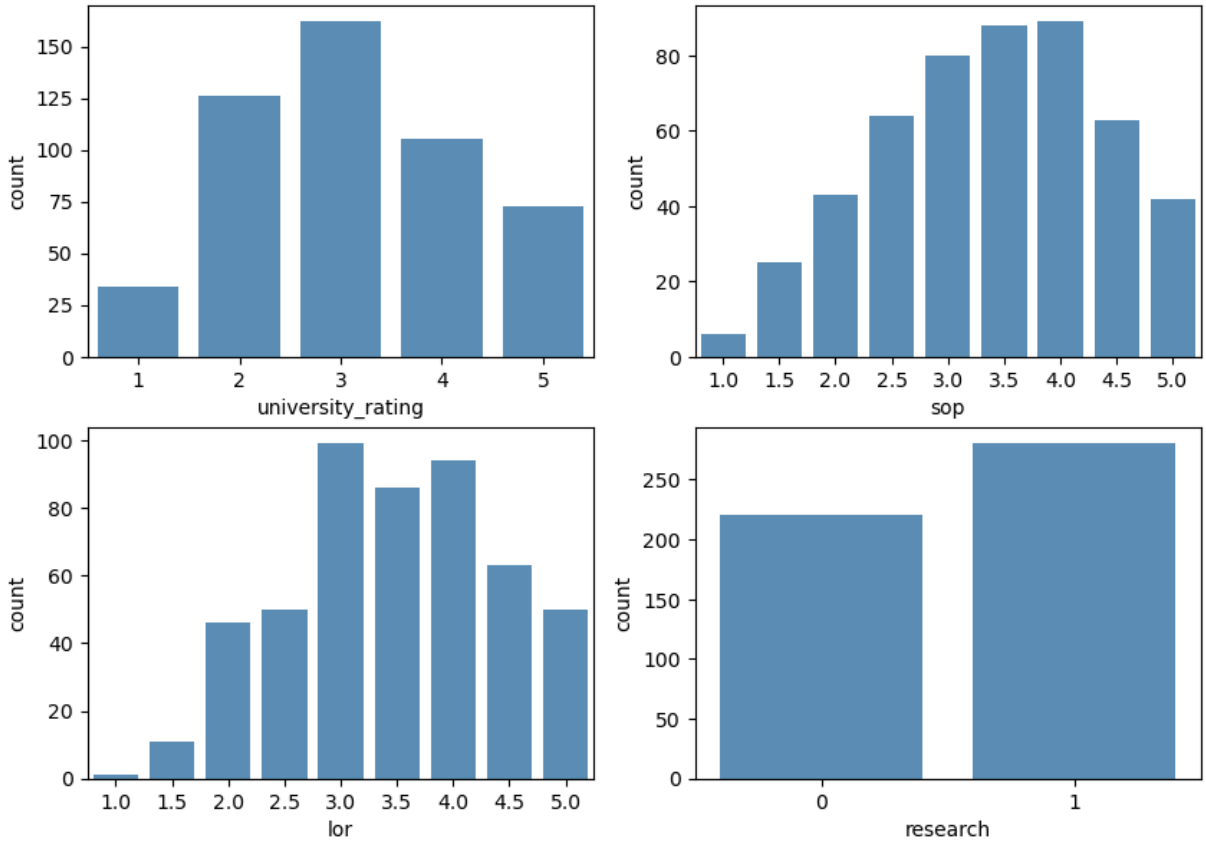
- There are No outliers found in the dataset.

In [20]:
```
# check unique values in categorical variables

unique_summary  = []
for col in cat_cols:
    unique_summary.append([col, df[col].nunique()])
pd.DataFrame(unique_summary, columns=["Column", "Unique Values"])
```

Out[20]:

	Column	Unique Values
0	university_rating	5
1	sop	9
2	lor	9
3	research	2

In [21]:
```
# countplots for categorical variables
cols, rows = 2, 2
fig, axs = plt.subplots(rows, cols, figsize=(10, 7))

index = 0
for row in range(rows):
    for col in range(cols):
        sns.countplot(x=cat_cols[index], data=df, ax=axs[row, col], alpha=0.8)
        index += 1

plt.show()
```

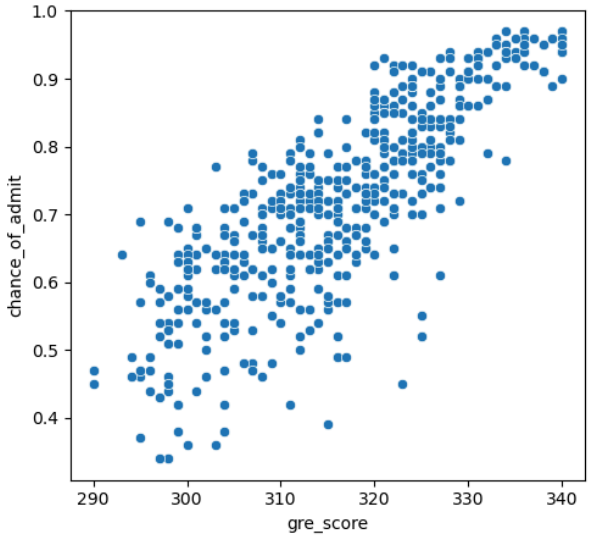


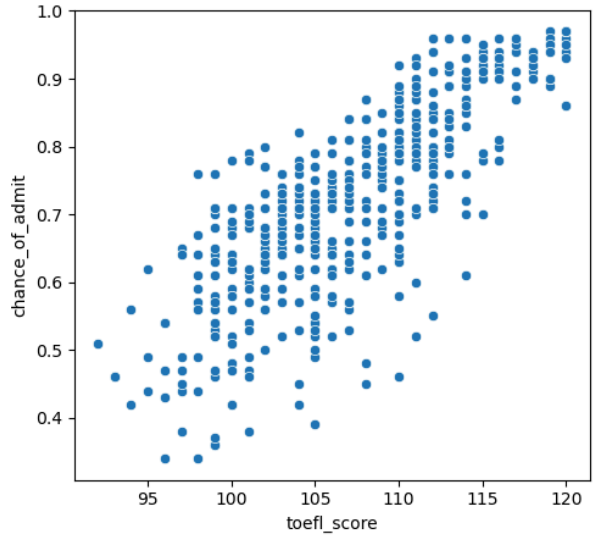
Categorical Columns

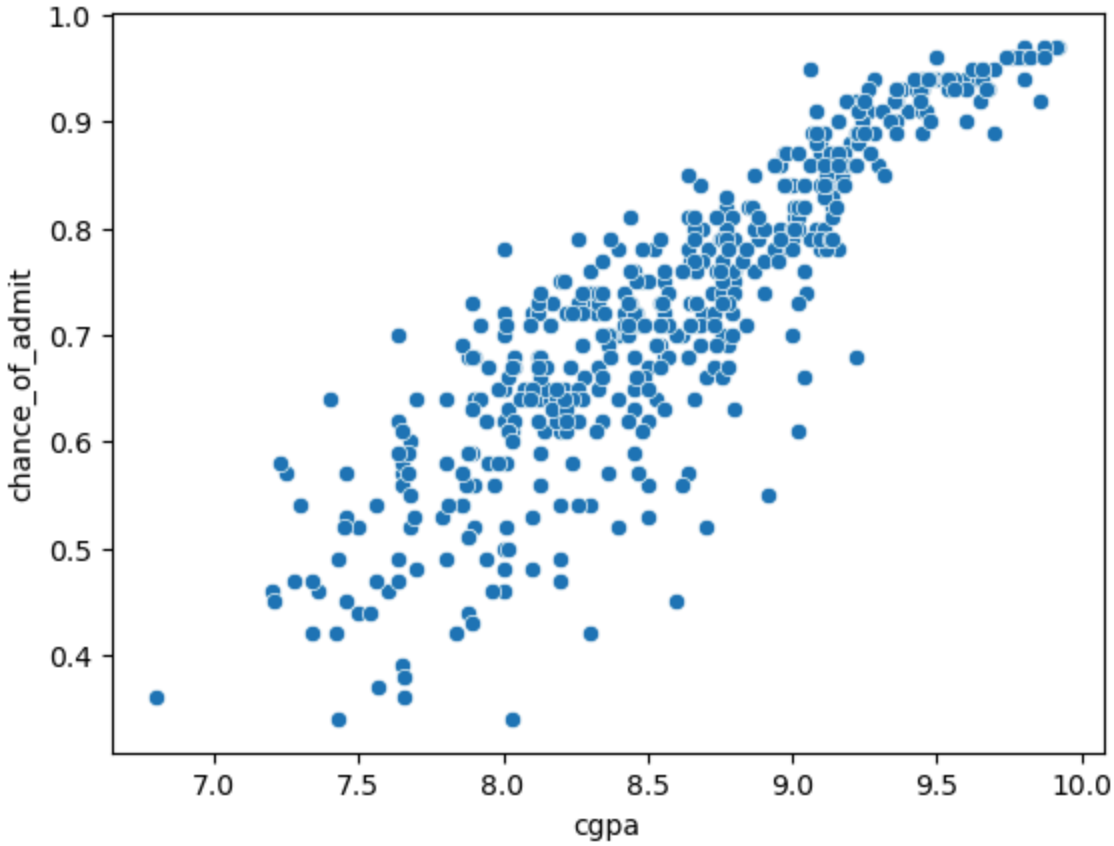
- **University Rating** → Values range from **1 to 5**, with **3** being the most frequent rating.
- **SOP (Statement of Purpose Strength)** → Values range from **1 to 5**.
- **LOR (Letter of Recommendation Strength)** → Values range from **1 to 5**.
- **Research** → Binary variable with **two unique values (0 = No, 1 = Yes)**.

Bivariate Analysis

In [22]:
```
# check relation between continuous variables & target variable
fig, axs = plt.subplots(1, 2, figsize=(12,5))

sns.scatterplot(x=num_cols[0], y=target, data=df, ax=axs[0])
sns.scatterplot(x=num_cols[1], y=target, data=df, ax=axs[1])
plt.show()
sns.scatterplot(x=num_cols[2], y=target, data=df)
plt.show()
```







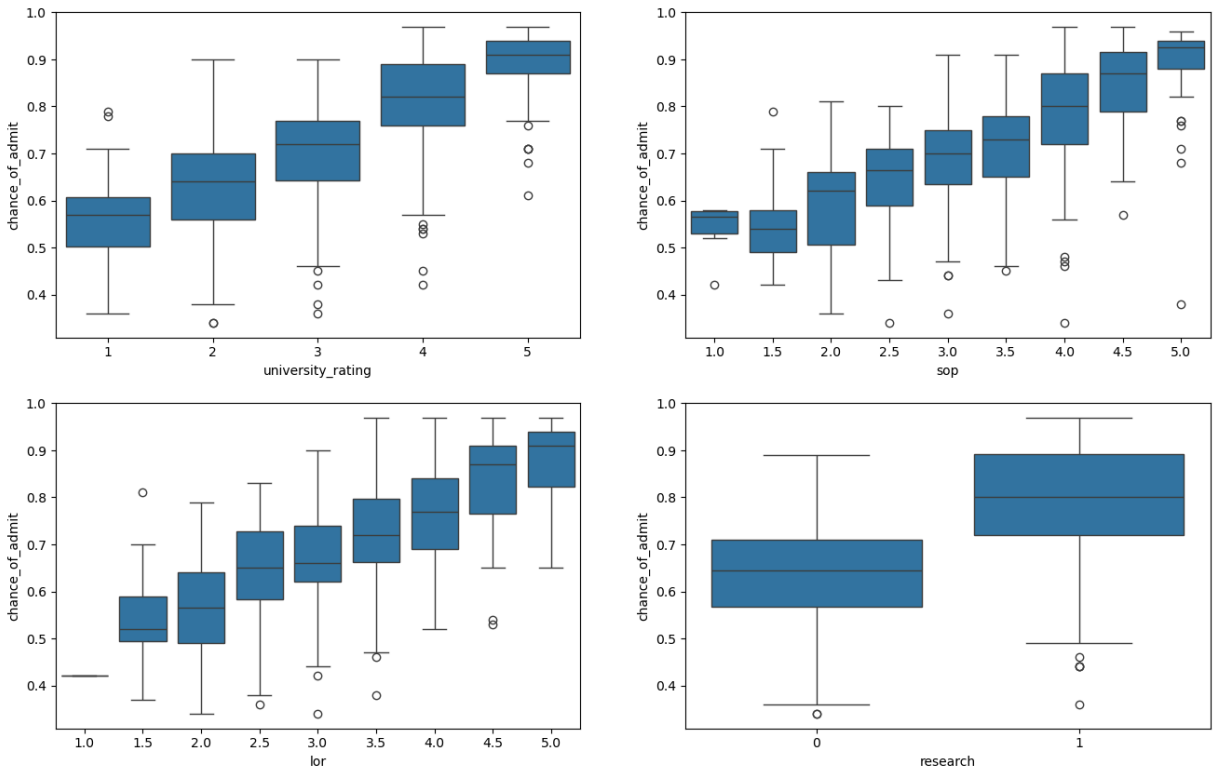
Numerical Columns

- **GRE Score** → Positively correlated with **Chance of Admit**.
- **TOEFL Score** → Positively correlated with **Chance of Admit**.
- **CGPA (Undergraduate GPA)** → Strong positive correlation with **Chance of Admit**.
- **Chance of Admit (Target Variable)** → Continuous variable ranging from 0 to 1.

Key Insight

- The numerical columns show a **linear relationship** with the target column (**Chance of Admit**).

In [23]:
```
rows, cols = 2,2
fig, axs = plt.subplots(rows, cols, figsize=(16,10))

index = 0
for row in range(rows):
    for col in range(cols):
        sns.boxplot(x=cat_cols[index], y=target, data=df, ax=axs[row,col])
        index += 1
```

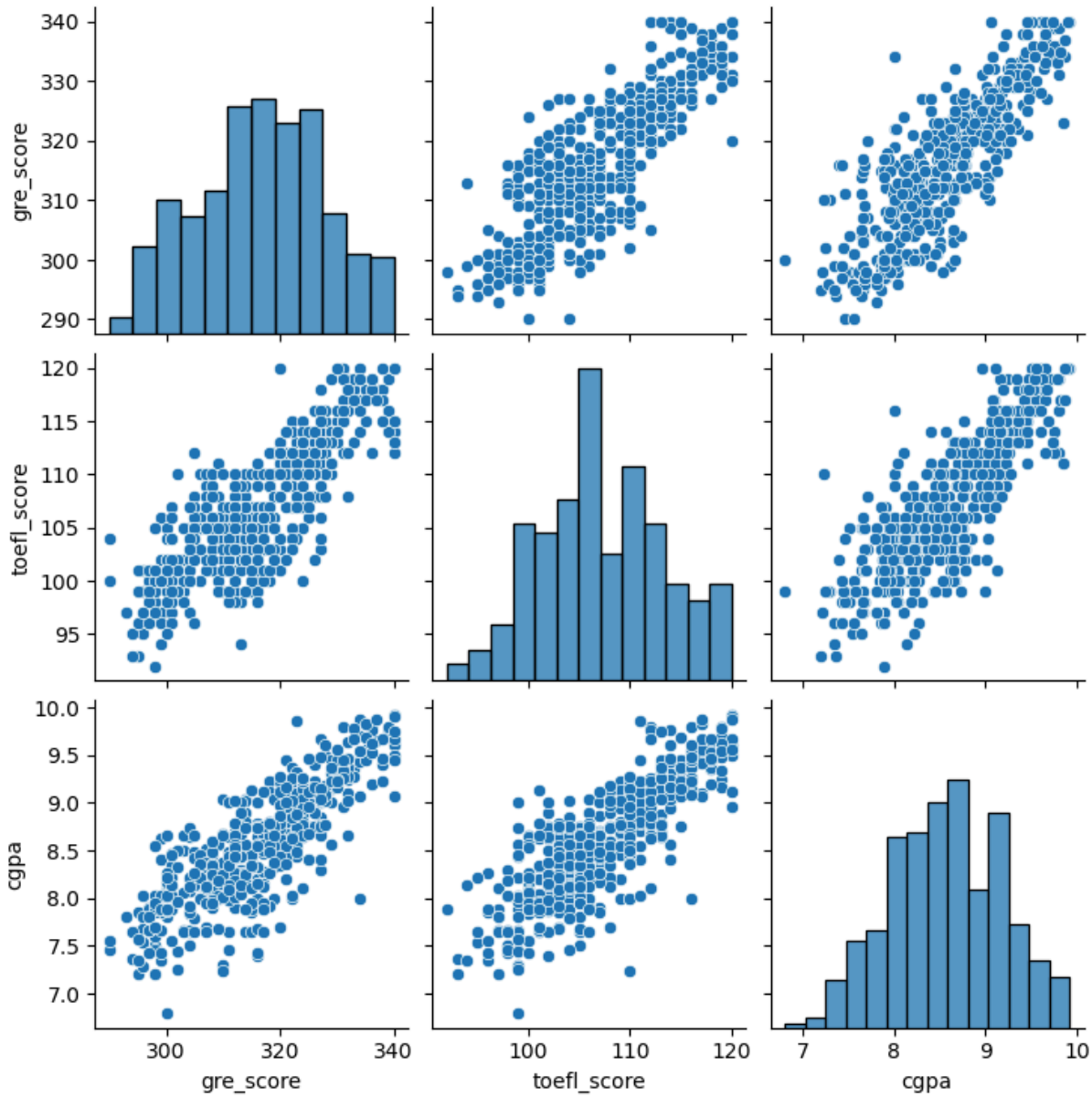


Insights from Categorical Variables

- **University Rating:** Higher university ratings are associated with an **increase in Chance of Admit**.
- **Research Experience:** Students with prior research experience have **higher chances of admission**.
- **LOR Strength:** Stronger Letters of Recommendation positively impact the **Chance of Admit**, similar to research experience and university rating.

Multivariate Analysis¶

In [24]:
```
sns.pairplot(df[num_cols])
plt.show()
```

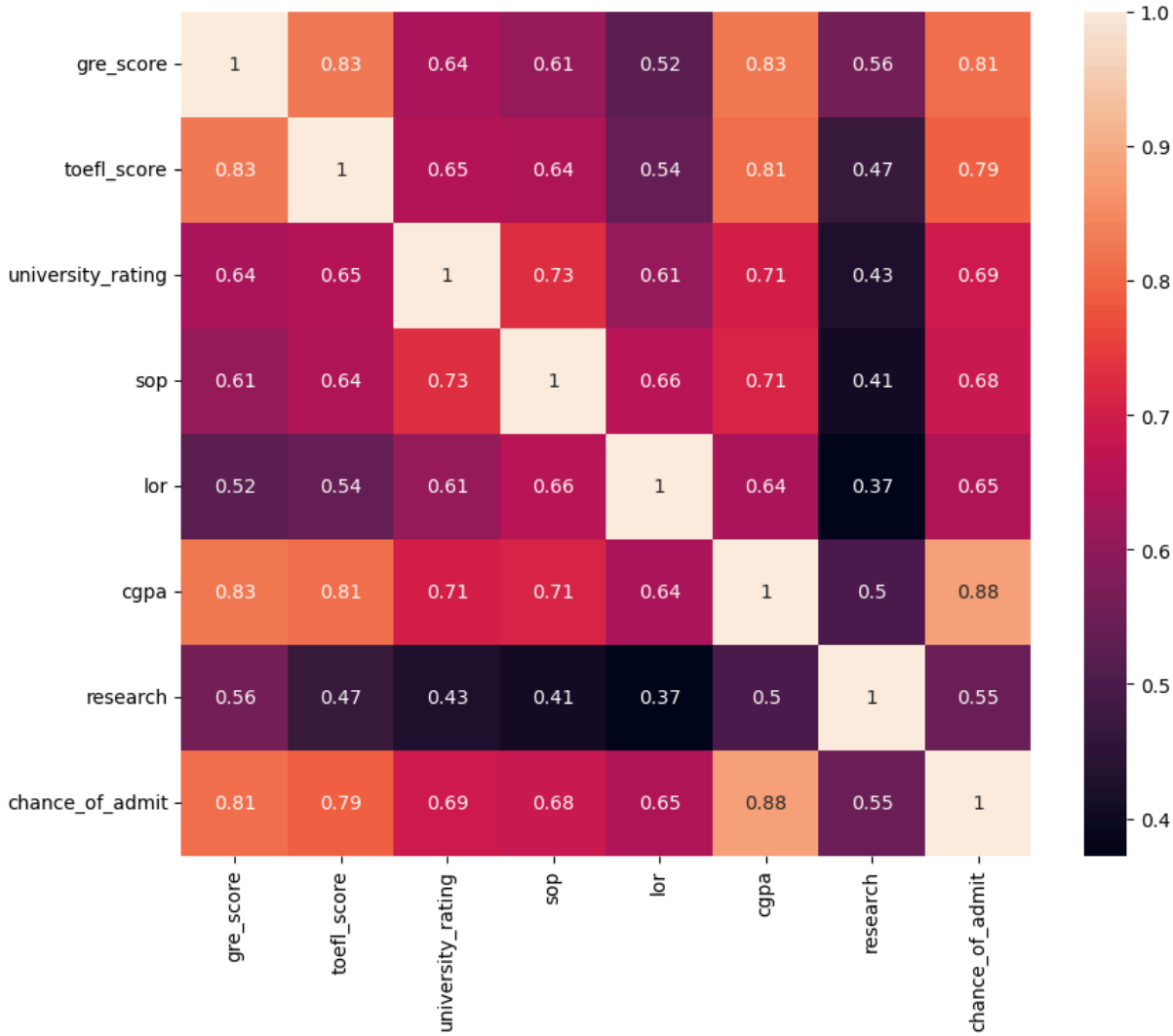


In [25]:
```
df.corr()
```

Out[25]:

	gre_score	toefl_score	university_rating	sop	lor	cgpa	res
gre_score	1.000000	0.827200	0.635376	0.613498	0.524679	0.825878	0.5
toefl_score	0.827200	1.000000	0.649799	0.644410	0.541563	0.810574	0.4
university_rating	0.635376	0.649799	1.000000	0.728024	0.608651	0.705254	0.4
sop	0.613498	0.644410	0.728024	1.000000	0.663707	0.712154	0.4
lor	0.524679	0.541563	0.608651	0.663707	1.000000	0.637469	0.3
cgpa	0.825878	0.810574	0.705254	0.712154	0.637469	1.000000	0.5
research	0.563398	0.467012	0.427047	0.408116	0.372526	0.501311	1.0
chance_of_admit	0.810351	0.792228	0.690132	0.684137	0.645365	0.882413	0.5

In [26]:
```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True)
plt.show()
```



Data Preprocessing

In [27]:
```
X = df.drop(columns=[target])
y = df[target]
```

In [28]:
```
# standardize the dataset
sc = StandardScaler()
X = sc.fit_transform(X)
```

In [29]:
```
# Performing the trian test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta
```

In [30]:
```
X_train
```

Out[30]:
```
array([[-0.83942999,  0.29757483, -0.09979274, ..., -0.52351283,
        -0.95404281,  0.88640526],
       [-0.83942999, -1.01912795, -0.09979274, ..., -0.52351283,
        -0.6230308 , -1.12815215],
       [ 1.19888185,  1.12051406, -0.09979274, ...,  1.09894429,
         1.09823165,  0.88640526],
       ...,
       [ 0.75577058, -0.03160087, -0.09979274, ...,  0.01730621,
         0.88307384,  0.88640526],
       [ 0.84439283,  0.62675052,  1.65095702, ...,  0.55812525,
         1.08168105,  0.88640526],
       [-1.45978576, -0.36077656, -1.85054249, ..., -1.60515091,
        -1.28505482, -1.12815215]])
```

In [31]:
```
X_test
```

Out[31]:
```
array([[-0.30769646, -0.19618871, -0.97516761, ..., -1.60515091,
        -0.24236699, -1.12815215],
       [-0.39631872, -0.03160087, -0.09979274, ..., -0.52351283,
        -0.19271519,  0.88640526],
       [ 1.99648213,  1.94345329,  1.65095702, ...,  0.55812525,
         1.85955927, -1.12815215],
       ...,
       [-1.10529675, -1.18371579,  0.77558214, ..., -1.06433187,
        -1.21885242, -1.12815215],
       [-0.57356322, -0.19618871,  0.77558214, ..., -1.06433187,
        -0.3582212 , -1.12815215],
       [ 1.55337086,  1.94345329,  1.65095702, ...,  1.09894429,
         1.85955927,  0.88640526]])
```

In [32]:
```
y_train
```

Out[32]:
```
13     0.62
61     0.47
298    0.90
39     0.48
360    0.85
       ...
255    0.79
72     0.93
396    0.84
235    0.88
37     0.58
Name: chance_of_admit, Length: 350, dtype: float64
```

In [33]:
```
y_test
```

Out[33]:
```
304    0.62
340    0.75
47     0.89
67     0.57
479    0.79
       ...
331    0.73
197    0.73
95     0.42
122    0.57
23     0.95
Name: chance_of_admit, Length: 150, dtype: float64
```

In [52]:
```python
# No of records afte performing the train test split
print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)
```

```
(350, 2) (350,)
(150, 2) (150,)
```

Model Building

In [35]:
```python
def adjusted_r2(r2, p, n):
    """
    n: no of samples
    p: no of predictors
    r2: r2 score
    """
    adj_r2 = 1 - ((1-r2)*(n-1) / (n-p-1))
    return adj_r2

def get_metrics(y_true, y_pred, p=None):
    n = y_true.shape[0]
    mse = np.sum((y_true - y_pred)**2) / n
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(y_true - y_pred))
    score = r2_score(y_true, y_pred)
    adj_r2 = None
    if p is not None:
        adj_r2 = adjusted_r2(score, p, n)

    res = {
        "mean_absolute_error": round(mae, 2),
        "rmse": round(rmse, 2),
        "r2_score": round(score, 2),
        "adj_r2": round(adj_r2, 2)
    }
    return res
```

In [36]:
```python
def train_model(X_train, y_train, X_test, y_test,cols, model_name="linear", alpha=1
    model = None
    if model_name == "lasso":
        model = Lasso(alpha=alpha)
    elif model_name == "ridge":
```

```
        model = Ridge(alpha=alpha)
    else:
        model = LinearRegression()

    model.fit(X_train, y_train)
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)
    p = X_train.shape[1]
    train_res = get_metrics(y_train, y_pred_train, p)
    test_res = get_metrics(y_test, y_pred_test, p)

    print(f"\n----   {model_name.title()} Regression Model  ----\n")
    print(f"Train MAE: {train_res['mean_absolute_error']} Test MAE: {test_res['mean
    print(f"Train RMSE: {train_res['rmse']} Test RMSE: {test_res['rmse']}")
    print(f"Train R2_score: {train_res['r2_score']} Test R2_score: {test_res['r2_sc
    print(f"Train Adjusted_R2: {train_res['adj_r2']} Test Adjusted_R2: {test_res['a
    print(f"Intercept: {model.intercept_}")
    #print(len(df.columns), len(model.coef_))
    coef_df = pd.DataFrame({"Column": cols, "Coef": model.coef_})
    print(coef_df)
    print("-"*50)
    return model
```

In [37]:

```
train_model(X_train, y_train, X_test, y_test,df.columns[:-1], "linear")
train_model(X_train, y_train, X_test, y_test,df.columns[:-1], "ridge")
train_model(X_train, y_train, X_test, y_test,df.columns[:-1], "lasso", 0.001)
```

```
----   Linear Regression Model  ----

Train MAE: 0.04 Test MAE: 0.04
Train RMSE: 0.06 Test RMSE: 0.06
Train R2_score: 0.82 Test R2_score: 0.82
Train Adjusted_R2: 0.82 Test Adjusted_R2: 0.81
Intercept: 0.724978121476996
              Column      Coef
0          gre_score  0.018657
1        toefl_score  0.023176
2  university_rating  0.011565
3                sop -0.000999
4                lor  0.012497
5               cgpa  0.064671
6           research  0.013968
--------------------------------------------------

----   Ridge Regression Model  ----

Train MAE: 0.04 Test MAE: 0.04
Train RMSE: 0.06 Test RMSE: 0.06
Train R2_score: 0.82 Test R2_score: 0.82
Train Adjusted_R2: 0.82 Test Adjusted_R2: 0.81
Intercept: 0.7249823645841696
              Column      Coef
0          gre_score  0.018902
1        toefl_score  0.023252
2  university_rating  0.011594
3                sop -0.000798
4                lor  0.012539
5               cgpa  0.064004
6           research  0.013990
--------------------------------------------------

----   Lasso Regression Model  ----

Train MAE: 0.04 Test MAE: 0.04
Train RMSE: 0.06 Test RMSE: 0.06
Train R2_score: 0.82 Test R2_score: 0.82
Train Adjusted_R2: 0.82 Test Adjusted_R2: 0.81
Intercept: 0.7249659139557142
              Column      Coef
0          gre_score  0.018671
1        toefl_score  0.022770
2  university_rating  0.010909
3                sop  0.000000
4                lor  0.011752
5               cgpa  0.064483
6           research  0.013401
--------------------------------------------------
```

Out[37]:

```
▾ Lasso
Lasso(alpha=0.001)
```

Key Observations from Model Evaluation

- Since the model is **not overfitting**, the results for **Linear Regression, Ridge Regression, and Lasso Regression** are almost the same.
- The **R^2 Score** and **Adjusted R^2 Score** are very close, which indicates that there are no unnecessary (redundant) independent variables in the dataset.

Linear Regression Model - Assumption Test¶

In [38]:
```
# Mutlicollinearity Check

def vif(newdf):
    # VIF dataframe
    vif_data = pd.DataFrame()
    vif_data["feature"] = newdf.columns

    # calculating VIF for each feature
    vif_data["VIF"] = [variance_inflation_factor(newdf.values, i)
                          for i in range(len(newdf.columns))]
    return vif_data
```

In [39]:
```
res = vif(df.iloc[:,:-1])
res
```

Out[39]:

	feature	VIF
0	gre_score	1308.061089
1	toefl_score	1215.951898
2	university_rating	20.933361
3	sop	35.265006
4	lor	30.911476
5	cgpa	950.817985
6	research	2.869493

In [40]:
```
# drop GRE Score and again calculate the VIF
res = vif(df.iloc[:, 1:-1])
res
```

Out[40]:

	feature	VIF
0	toefl_score	639.741892
1	university_rating	19.884298
2	sop	33.733613
3	lor	30.631503
4	cgpa	728.778312
5	research	2.863301

In [41]:

```
#drop TOEFL Score and again calculate the VIF
res = vif(df.iloc[:,2:-1])
res
```

Out[41]:

	feature	VIF
0	university_rating	19.777410
1	sop	33.625178
2	lor	30.356252
3	cgpa	25.101796
4	research	2.842227

In [42]:

```
# Now lets drop the SOP and again calculate VIF
res = vif(df.iloc[:,2:-1].drop(columns=['sop']))
res
```

Out[42]:

	feature	VIF
0	university_rating	15.140770
1	lor	26.918495
2	cgpa	22.369655
3	research	2.819171

In [43]:

```
# lets drop the LOR as well
newdf = df.iloc[:,2:-1].drop(columns=['sop'])
newdf = newdf.drop(columns=['lor'], axis=1)
res = vif(newdf)
res
```

Out[43]:

	feature	VIF
0	university_rating	12.498400
1	cgpa	11.040746
2	research	2.783179

In [44]:

```
# drop the University Rating
newdf = newdf.drop(columns=['university_rating'])
res = vif(newdf)
res
```

Out[44]:

	feature	VIF
0	cgpa	2.455008
1	research	2.455008

In [45]:

```
df.columns
```

Out[45]:

```
Index(['gre_score', 'toefl_score', 'university_rating', 'sop', 'lor', 'cgpa',
       'research', 'chance_of_admit'],
      dtype='object')
```

In [46]:

```
# now again train the model with these only two features
X = df[['cgpa', 'research']]
sc = StandardScaler()
X = sc.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta
```

In [47]:

```
model = train_model(X_train, y_train, X_test, y_test, ['CGPA', 'Research'], "linear
train_model(X_train, y_train, X_test, y_test, ['CGPA', 'Research'], "ridge")
train_model(X_train, y_train, X_test, y_test, ['CGPA', 'Research'], "lasso", 0.001)
```

```
----   Linear Regression Model  ----

Train MAE: 0.05 Test MAE: 0.05
Train RMSE: 0.06 Test RMSE: 0.07
Train R2_score: 0.78 Test R2_score: 0.81
Train Adjusted_R2: 0.78 Test Adjusted_R2: 0.81
Intercept: 0.7247774222727991
     Column      Coef
0      CGPA  0.112050
1  Research  0.020205
--------------------------------------------------

----   Ridge Regression Model  ----

Train MAE: 0.05 Test MAE: 0.05
Train RMSE: 0.06 Test RMSE: 0.07
Train R2_score: 0.78 Test R2_score: 0.81
Train Adjusted_R2: 0.78 Test Adjusted_R2: 0.81
Intercept: 0.7247830300095277
     Column      Coef
0      CGPA  0.111630
1  Research  0.020362
--------------------------------------------------

----   Lasso Regression Model  ----

Train MAE: 0.05 Test MAE: 0.05
Train RMSE: 0.06 Test RMSE: 0.07
Train R2_score: 0.78 Test R2_score: 0.81
Train Adjusted_R2: 0.78 Test Adjusted_R2: 0.81
Intercept: 0.7247713356661623
     Column      Coef
0      CGPA  0.111344
1  Research  0.019571
--------------------------------------------------
```

Out[47]:

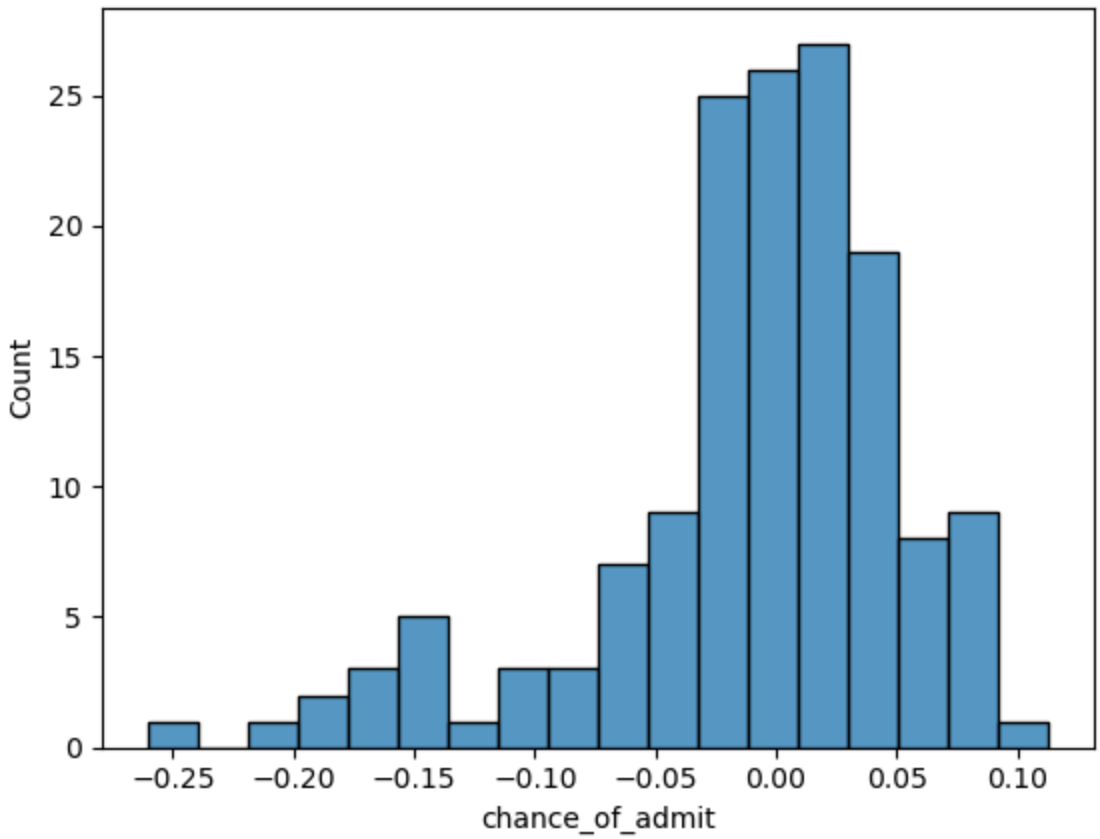
```
Lasso
Lasso(alpha=0.001)
```

Model Evaluation Observations

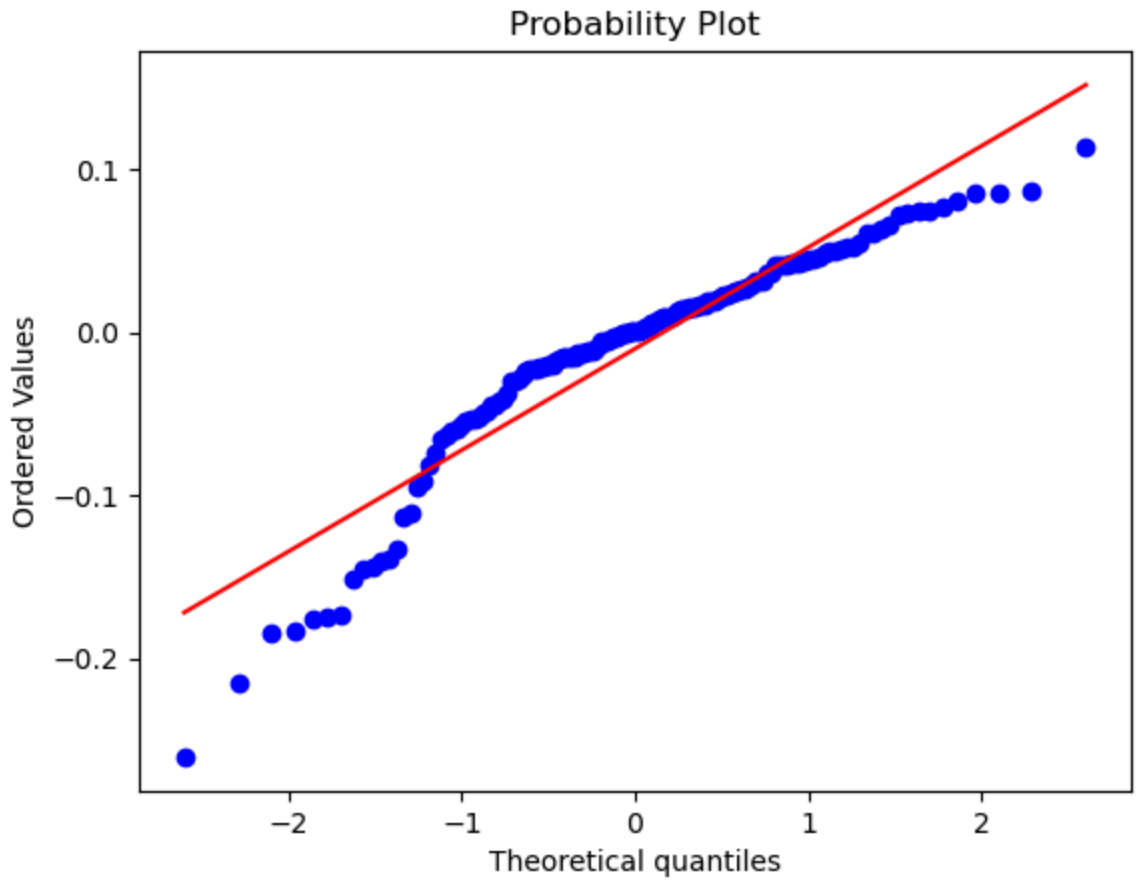
- After removing collinear features using **VIF** and keeping only two features, the **R^2 Score** and **Adjusted R^2** remain the same on the testing dataset. This shows that the removed variables were redundant and not adding predictive power.
- From the **RMSE**, it is clear that the **Mean of Residuals** is almost zero. This validates that the errors are unbiased and centered around zero.
- From **EDA**, it is evident that the independent variables are **linearly dependent** on the target variable, supporting the assumption of linear regression.

Normality of Residuals

In [49]:
```
y_pred = model.predict(X_test)
residuals = (y_test - y_pred)
sns.histplot(residuals)
plt.show()
```

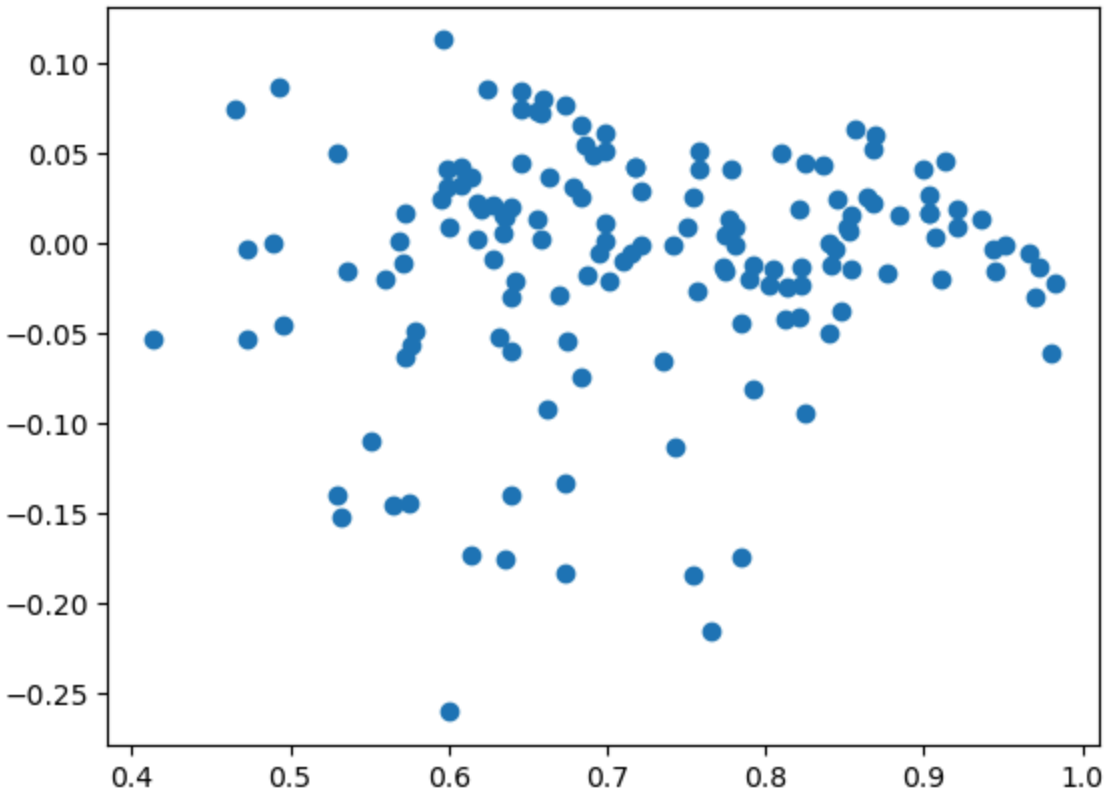


In [50]:
```
stats.probplot(residuals, plot=plt)
plt.show()
```



Test for Homoscedasticity

In [51]:
```
plt.scatter(y_pred, residuals)
plt.show()
```



Homoscedasticity Check

- Upon examining the residual plot, it is evident that the points **do not form a cone-shaped pattern**.
- This indicates that **homoscedasticity is not present** in the data (i.e., the variance of residuals is not constant).

Summary Table

Aspect	Key Findings / Results
Important Predictors	CGPA, Research
Most Influential Feature	CGPA
Other Relevant Features	University Rating, LOR Strength
Multicollinearity	Present, resolved using VIF
Relationship with Target	Numerical: Linear (GRE, TOEFL, CGPA)
	Categorical: Higher Univ Rating, Research, LOR ↑ Admit
Model Assumptions	Linearity satisfied, Residuals mean ~0
RMSE (Test Data)	0.07
MAE (Test Data)	0.05

Aspect	Key Findings / Results
R^2 Score (Test Data)	0.81
Adjusted R^2 (Test Data)	0.81
Recommendation Summary	Focus on CGPA and Research; improve LOR & research exposure

Insights

1. **Multicollinearity:** Present in the dataset, but after removing collinear features using VIF, only **CGPA** and **Research** remain important predictors.
2. **Numerical Features:** GRE, TOEFL, and CGPA show a **linear relationship** with the target variable (Chance of Admit). CGPA is the strongest predictor.
3. **Categorical Features:**
   - **University Rating:** Higher ratings increase the chance of admission.
   - **Research Experience:** Students with prior research experience have higher admission chances.
   - **LOR Strength:** Strong letters of recommendation positively impact the target variable.
4. **Target Variable Distribution:** The target variable is continuous (0–1), with no nulls or duplicates in the dataset.
5. **Model Performance:** Linear Regression, Ridge, and Lasso models produce similar results, indicating **no overfitting**. The final model achieves:
   - **RMSE:** 0.07
   - **MAE:** 0.05
   - **R^2 Score:** 0.81
   - **Adjusted R^2:** 0.81

Recommendations

1. **Key Predictors:** Only **CGPA** and **Research Experience** are necessary for predicting Chance of Admit. Other features can be considered optional.
2. **Prioritize CGPA:** CGPA has the strongest impact and should be emphasized when advising students.
3. **Model Reliability:** The current regression model is robust, satisfies linearity assumptions, and can be reliably used for prediction.

In [ ]: